

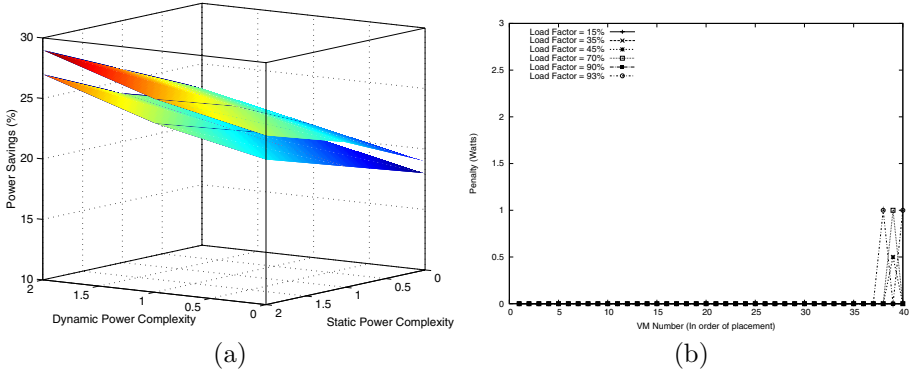


Fig. 10. (a) Power Savings of *mPP* and *mPPH* in comparison to Static Placement with increasing heterogeneity. (b) Penalty incurred because of the Ordering Assumption.

additive with increase in static or dynamic complexity. Hence, the proposed algorithms are capable of working even better in heterogeneous clusters, thus establishing their efficacy in a wide variety of settings.

Our algorithms are based on the assumption that for each application being placed, we can characterize the incremental power drawn by it at various workloads on all the server platforms. We can then use this incremental power table to decide which server to place the application on, during dynamic application placement. We now investigate the practicality of this assumption. In order to study this, we looked at a random sample of 100 points from this selection trace log and measured the incremental power drawn due to our selection and the incremental power drawn by selecting any other server. Fig. 10(b) shows the accuracy of our estimate in terms of the penalty accrued due to wrong selection for each placed application. If we made the right selection, we get a penalty of 0 whereas if there was another server with less incremental power, we incur a penalty equal to the difference. We observe that the *Ordering* property holds for most of the applications with a few errors resulting in an error of 1 watts or less. As compared to the total power consumption of about 750W, this error is insignificant. Further, we observe that the property does not hold only for the last few applications being placed. Hence, any errors made do not get cascaded to other applications, as most of them were already placed. This is a direct result of the fact that we chose to adapt *FFD*, which leads to unbalanced load. Hence, for most of the comparisons that used the *Ordering* property, we compared an idle server with a loaded server, and very clearly the loaded server was selected because of the huge static power cost. Hence, our local search based on the *Ordering* property worked close to optimal. Further, a closer look at the plots reveal that we pay the penalty only when the overall load approaches 50% or 75%. This is because in these cases all the servers that were loaded were close